Vision and Scope Document

for

Програмна система для моніторингу та обслуговування сенсорних постів «SensorDesk»

Version 1.0 approved

Prepared by Kulinich Oleksandr, PZPI-24-6

Харківський національний університет радіоелектроніки

07.10.2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents 2
Revision History 2
1. Бізнес вимоги 1
    1.1. Аналіз проблеми, яку вирішує програмна система 1
    1.2. Аналіз аналогів та способи монетизації 1
2. Бачення рішення 2
    2.1. Опис призначення системи та головних можливостей 2
    2.2. Опис бізнес-логіки системи 2
3. Обсяг і обмеження 3
    3.1. Опис функціональності серверної частини та IoT-клієнта 3
    3.2. Опис функціональності веб-клієнта та мобільного клієнта 4
4. Бізнес контекст 4
    4.1. Зацікавлені сторони 4
    4.2. Опис програмних засобів та програмних інструментів 5
Висновки 5

Revision History

Name	Date	Reason For Changes	Version
Кулініч Олександр	08.10.2026	Перша версія документа	1.0

1. Бізнес вимоги

SensorDesk необхідна, бо відмова сенсорного поста залишається непоміченою / поміченою з критичними затримками, а ремонт не контролюється від заявки до підтвердженого відновлення.

1.1. Аналіз проблеми, яку вирішує програмна система

Цивільні об'єкти (офісні й складські будівлі, серверні приміщення, комунальна та енергетична інфраструктура, екологічний моніторинг) використовують сенсорні пости. Сенсорний пост - це автономний вузол на базі мікроконтролера з датчиками температури та напруги живлення й модулем зв'язку. Коли пост втрачає живлення, перегрівається, губить мережу, або виходить з ладу, система моніторингу втрачає свої очі.

На практиці обслуговування ведеться доволі неформально: заявки губляться в месенджерах і таблицях, немає єдиної черги й відповідальних, техніки виїжджають без історії поста, а після ремонту ніхто не перевіряє, що пост справді працює. Через це трапляються повторні виїзди й затяжні простої.

Модель «сервіс за підпискою з обслуговуванням» запозичено з стовпця оригінальних проєктів (IMPACT Drones, Air Defense as a Service). Реалізація повністю цивільна: це система обліку й обслуговування постів на макеті.

Цільова аудиторія – малі та середні організації з територіально розподіленими об'єктами. Для масового використання передбачено багатоорендну архітектуру: одна інсталяція обслуговує багатьох замовників, а кількість постів може зростати від десятків до тисяч за рахунок горизонтального масштабування серверної частини.

1.2. Аналіз аналогів та способи монетизації

Система закриває прогалину між моніторингом і обслуговуванням: наявні рішення або показують відмови, або ведуть заявки, але рідко поєднують це з перевіркою відновлення за самотестом пристрою. Аналоги:

- **CMMS (UpKeep, Fiix, Limble CMMS).** Заявки, планові ТО, мобільні застосунки, QR-коди обладнання. Як правило, немає вбудованого розбору телеметрії сенсорних постів і автоматичної перевірки відновлення.
- **ITSM і Field Service (Jira Service Management, ServiceNow).** Дуже функціональні, але універсальні, складні в налаштуванні й дорогі для малих команд.
- **Моніторинг (Zabbix, Grafana з алертами).** Показують відмови й графіки, але не ведуть візити техніків, сервісні вікна та закриття заявок.
- **Платформи керування пристроями (Memfault, AWS IoT Device Management).** Орієнтовані на прошивки й парк пристроїв, а не на польове обслуговування людьми.

Орієнтовне порівняння за ключовими критеріями:

Критерій	**CMMS**	**ITSM**	**Моніторинг**	**Керування пристроями**	**SensorDesk**
Телеметрія постів	Ні	Ні	Так	Так	Так
Автоматичні заявки за телеметрією	Частково	Частково	Ні	Ні	Так
Візити та сервісні вікна	Так	Так	Ні	Ні	Так
Перевірка відновлення самодіагностикою	Ні	Ні	Ні	Частково	Так
Автономний режим техніка	Частково	Частково	Ні	Ні	Так
Простота налаштування для малих команд	Частково	Ні	Частково	Ні	Так

Переваги SensorDesk: це повний ланцюг «телеметрія > заявка > візит > автоматична перевірка працездатності», вузька предметна область з простим налаштуванням та модуль штучного інтелекту, що готує чернетку плану діагностики за інструкцією конкретного поста.

Монетизація (теоретична):

- підписка SaaS залежно від кількості постів, тарифи «Базовий», «Стандарт» і «SLA» із гарантованим часом реакції;
- платний модуль LLM-асистента діагностики;
- продаж або оренда комплектів вузла з прошивкою;
- послуги впровадження та інтеграції з системами замовника;
- платний доступ до API для інтеграцій.

2. Бачення рішення

SensorDesk має стати єдиним робочим місцем, де відмова поста перетворюється на заявку й завершується підтвердженим відновленням.

2.1. Опис призначення системи та головних можливостей

Для організацій, які експлуатують сенсорні пости на цивільних об’єктах, SensorDesk є програмною системою, що поєднує телеметрію постів, заявки на обслуговування, візити техніків і автоматичну перевірку відновлення. На відміну від таблиць і месенджерів, система гарантує відповідальних, історію та доказ того, що пост працює. Усі рішення про виконання робіт ухвалює людина, а LLM лише готує чернетки.

Головні функціональні можливості:

1. **MF-1. Реєстр об’єктів і постів.** Паспорт поста, інструкція, профіль порогів, історія обслуговування.
2. **MF-2. Приймання телеметрії.** Доступність, температура, живлення; пакетна відправка по HTTPS, унікальний ID пакета, ідемпотентна обробка повторів.
3. **MF-3. Правила відмов і автоматичні заявки.** Пост недоступний довше за задану кількість хвилин, напруга нижча за поріг або температура вища за поріг створюють заявку; повторні події того самого поста зливаються в одну відкриту заявку.
4. **MF-4. Життєвий цикл заявки.** Нова > Призначена > Запланована > У роботі > Очікує перевірки > Виконана; невдала перевірка повертає заявку «У роботі». Пріоритет залежить від критичності об’єкта й типу відмови, для кожного пріоритету задано цільовий час реакції.
5. **MF-5. Призначення й планування.** Техніка обирають з урахуванням сервісного вікна об’єкта та завантаження; одну заявку не можна призначити двом виконавцям одночасно.
6. **MF-6. Мобільний застосунок техніка.** Маршрут візитів, QR поста, BLE-діагностика, фото ремонту, робота офлайн із подальшою синхронізацією, push-сповіщення.
7. **MF-7. Самотест поста.** Послідовність перевірок (ідентичність, температура, напруга, зв’язок, індикація) з результатом PASS, WARN або FAIL і виміряними пороговими значеннями.
8. **MF-8. Перевірка відновлення.** Заявку не можна закрити вручну: потрібні успішний самотест поточного візиту, стабільна телеметрія заданий час і підтвердження техніка з фото.
9. **MF-9. LLM-чернетка плану діагностики та результату діагностики.** Пошук в інструкції конкретного поста й чернетка плану з посиланнями на фрагменти; виконання лише після погодження людиною.
10. **MF-10. Звіти та показники.** Простої, час реакції, повторні звернення, виконання заявок.
11. **MF-11. Ролі та аудит.** Адміністратор, диспетчер, технік, замовник (перегляд); усі зміни стану й дії користувачів у журналі.
12. **MF-12. Адміністрування системи.** Керування користувачами та даними, резервне копіювання й відновлення, експорт та імпорт даних і налаштувань, керування сертифікатами.
13. **MF-13. Інтернаціоналізація та локалізація. І**нтерфейс українською та англійською мовами; часові пояси, формати дати й часу, одиниці вимірювання, порядок сортування, кодування UTF-8.
14. **MF-14. Захист персональних даних.** Відповідність вимогам GDPR: мінімізація даних, право на доступ, виправлення та видалення, шифрування передавання й зберігання.

2.2. Опис бізнес-логіки системи

Бізнес-логіка побудована навколо заявки на обслуговування, яка народжується з телеметрії й закривається лише після перевірки відновлення.

Основні сутності: об’єкт, пост, заявка, сервісне вікно, технік, візит, інструкція поста, результат самодіагностики, подія телеметрії.

Життєвий цикл заявки: Нова > Призначена > Запланована > У роботі > Очікує перевірки > Закрита. Якщо перевірка не пройдена, заявка повертається в стан «У роботі».

Правила:

- Заявка створюється автоматично, якщо пост недоступний довше за задану кількість хвилин, напруга нижча за поріг або температура вища за поріг. Повторні події того самого поста зливаються в одну відкриту заявку.
- Пріоритет залежить від критичності об’єкта й типу відмови; для кожного пріоритету задано цільовий час реакції, і система відстежує його дотримання.
- Техніка призначають з урахуванням сервісного вікна об’єкта й завантаження; одну заявку не можна призначити двом виконавцям одночасно.
- Заявку не можна закрити вручну. Для закриття потрібні одночасно: успішний самотест, прив’язаний до поточного візиту, стабільна телеметрія протягом заданого часу та підтвердження техніка з фото.
- LLM лише готує чернетку плану діагностики за інструкцією поста; рішення про виконання приймає людина, а пропозиції зберігаються в журналі.
- Права доступу залежать від ролі: адміністратор, диспетчер, техник, замовник (лише перегляд). Усі зміни стану й дії користувачів записуються в журнал аудиту.

3. Обсяг і обмеження

Перший реліз охоплює повний наскрізний сценарій від відмови до верифікованого закриття на стенді з кількома постами; решта відкладається.

3.1. Опис функціональності серверної частини та IoT-клієнта

Серверна частина (Web API).

- автентифікація й авторизація за ролями (JWT);
- CRUD для об’єктів, постів, заявок, виконавців, сервісних вікон і візитів;
- приймання пакетів телеметрії з унікальним ID та ідемпотентною обробкою повторів;
- зберігання сирих вимірювань, добові агрегати, розрахунок простоїв;
- рушій правил: виявлення відмов, створення й дедуплікація заявок, контроль часу реакції;
- перевірка відновлення за самотестом і стабільною телеметрією;
- push-сповіщення, журнал аудиту, звіти, документація API (OpenAPI);
- LLM-модуль: чернетка плану діагностики з посиланнями на фрагменти інструкції.

- адміністрування: керування користувачами, ролями й даними; автоматичне та ручне резервне копіювання й відновлення бази даних; експорт та імпорт даних і налаштувань (JSON, CSV);
- інтернаціоналізація та локалізація: українські й англійські ресурси повідомлень; зберігання часу в UTC із перетворенням у часовий пояс користувача; формат ISO 8601; температура у °C або °F і відстані у кілометрах або милях за налаштуванням; сортування рядків за правилами обраної мови.
- захист персональних даних: мінімізація даних користувачів (ПІБ, контактні дані), згода на обробку, експорт даних суб'єкта, виправлення та видалення (анонімізація зі збереженням запису аудиту), строки зберігання, хешування паролів, TLS для всього трафіку, розмежування даних різних замовників.

Вимоги: бізнес-логіка відокремлена від транспорту, ключові правила покриті автотестами, вхідні дані валідуються, часові зони обробляються коректно.

IoT-клієнт (пост на ESP32, C++).

- періодичні вимірювання температури й напруги з перевіркою діапазонів і часовими мітками;
- heartbeat для контролю доступності;
- самотест за кнопкою, командою з телефона по BLE або з сервера: ідентичність і час, температура (відповідь датчика, кілька відліків, діапазон і розкид), напруга (відповідь модуля, діапазон, шум), зв'язок (Wi-Fi, RSSI, запит /health, BLE), буфер (тестовий запис і читання, вільне місце на флеш пам'яті посту), індикація;
- кожна перевірка дає PASS, WARN або FAIL з виміряним значенням, загальний вердикт FAIL при провалі критичної перевірки;
- результат має унікальний testId та прив'язаний до visitId візиту, який телефон передає по BLE;
- локальний буфер на випадок обриву зв'язку й повторна відправка після відновлення;
- BLE-сервіс для телефона: стан, запуск самотесту, налаштування; світлодіодна індикація.

Вимоги: стабільна робота без втручання, обмежене енергоспоживання, захист від втрати даних, пороги з профілю поста.

3.2. Опис функціональності веб-клієнта та мобільного клієнта

Веб-клієнт (через API).

- панель диспетчера: черга заявок, фільтри за пріоритетом і станом;
- призначення техніка та планування за сервісними вікнами;
- картки постів: паспорт, графіки телеметрії, історія обслуговування;
- показники: час реакції, простої, повторні звернення;
- керування користувачами й ролями, перегляд журналу аудиту.
- сторінка адміністратора: резервні копії, експорт та імпорт, сертифікати;

- перемикання мови (українська, англійська), часового поясу й одиниць вимірювання.

Вимоги: адаптивний інтерфейс, розмежування доступу за ролями, зрозумілі статуси та повідомлення про помилки.

Мобільний клієнт (Android, Kotlin).

- перелік візитів і маршрут техніка;
- сканування QR поста, відкриття картки й інструкції;
- BLE-діагностика та запуск самотесту на місці;
- фото ремонту й підтвердження виконання;
- офлайн-режим: локальне збереження й синхронізація після відновлення мережі;
- push-сповіщення про нові й змінені заявки.
- українська та англійська локалізація.

Вимоги: коректна робота без мережі, безпечне зберігання токена, мінімум дій для підтвердження візиту.

3.3. Наступні релізи, обмеження та виключення

Наступні релізи: MQTT як додатковий транспорт, бездротове оновлення прошивки постів, клієнт для iOS, прогнозне обслуговування за трендами телеметрії.

Виключення: система не є засобом безпеки чи аварійного оповіщення в реальному часі. Система не керує виконавчими пристроями. Платіжна підсистема не реалізується (монетизація теоретична).

4. Бізнес контекст

Проєкт виконує один розробник; нижче наведено зацікавлені сторони, строки та програмні засоби.

4.1. Зацікавлені сторони

Stakeholder	Major Value	Attitudes	Major Interests	Constraints
Замовник	Менше простоїв, прозорий облік обслуговування	Зацікавлений, чутливий до вартості підписки	Доказ відновлення, звіти, SLA	Бюджет на підписку й обладнання
Диспетчер	Єдина черга заявок замість листування	Сприйнятливий, очікує швидкий інтерфейс	Пріоритети, призначення, контроль строків	Працює переважно з веб-клієнта

Технік	Повна картка поста на місці, менше повторних виїздів	Сприйнятливий, якщо застосунок простий і працює офлайн	QR, BLE-діагностика, фото, офлайн-режим	Нестабільна мережа, обмежений час на візит
Адміністратор системи	Керування користувачами, порогами й інструкціями	Нейтральний	Ролі, налаштування, аудит	Потрібні прості налаштування

4.2. Строки та пріоритети проєкту

Етап	Орієнтовний строк
Vision & Scope	23.10.2026
база даних, API серверної частини	14.11.2026
бізнес-логіка та адміністрування сервера	20.11.2026
бізнес-логіка та адміністрування сервера	04.12.2026
розгортання та демонстрація	25.12.2026

4.3. Опис програмних засобів та програмних інструментів

Програмні засоби та інструменти:

- сервер: Java 21, Spring Boot, Spring Security (JWT), PostgreSQL, Flyway, Gradle, OpenAPI, JUnit;
- LLM: інтеграція через API (модель із OpenAI-сумісним інтерфейсом, або google-genai);
- IoT: ESP32, C++, BLE, HTTPS, датчик напруги, термодатчик;
- веб-клієнт: TypeScript, React, Vite;
- мобільний клієнт: Kotlin, Android Studio, Jetpack Compose, Room, WorkManager, CameraX, Firebase Cloud Messaging, Google ML Kit;
- інфраструктура й проєктування: Git і GitHub, Docker, draw.io, Figma.

Висновки

У ході роботи описано проблему необлікованого обслуговування сенсорних постів, проаналізовано аналоги, визначено призначення та бізнес-логіку SensorDesk, функції серверної частини, IoT-клієнта, веб-клієнта та мобільного клієнта, обмеження а також зацікавлені сторони, строки та технології. Результат стане основою для проєктування системи. Відеозапис з описом системи наведено в додатку А.

Перелік джерел

1. UpKeep. URL: https://upkeep.com
2. Fiix. URL: https://fiixsoftware.com
3. Limble CMMS. URL: https://limble.com/
4. Jira Service Management. URL: https://www.atlassian.com/software/jira/service-management
5. Zabbix. URL: https://www.zabbix.com
6. Grafana. URL: https://grafana.com
7. Memfault. URL: https://memfault.com
8. AWS IoT Device Management. URL: https://aws.amazon.com/iot-device-management/

Додаток А
Відеозапис

Відеозапис доповіді до лабораторної роботи №1 розміщено на YouTube:
https://youtu.be/69vzmVjgnjk

00:00 Вступ
00:25 1.1 Аналіз проблеми, яку вирішує програмна система
01:44 1.2 Аналіз аналогів та способи монетизації
04:15 2.1 Опис призначення системи та головних можливостей
04:55 2.2 Опис бізнес-логіки системи
07:10 3.1 Опис функціональності серверної частини та IoT-клієнта
09:09 3.2 Опис функціональності веб-клієнта та мобільного клієнта
10:24 4.3 Опис програмних засобів та програмних інструментів
11:08 Завершення